INFORME TÈCNIC

PAC1
SUPERVISOR DE PROCESSOS

Mòdul: MP0490 · Programació de serveis i processos

Autor/a: Aitor Ocaña Navarro

Data de lliurament: 26 d'octubre de 2026

Repo: https://github.com/aitorocana/ProgramacioProcessos_pt1

1. Objectiu de la pràctica

L'objectiu és crear un programa Java que actuï com a SUPERVISOR. El programa inicia processos fills independents (WORKER, PRODUCER, CONSUMER i ECHO), els envia dades per stdin, llegeix stdout i stderr, n'obté el resultat i els atura si superen un límit de temps.

2. Arquitectura del programa

El projecte està dividit en tres fitxers. Cada fitxer té una funció concreta:

Fitxer	Responsabilitat
CreacioProcessos.java	Conté main(), el supervisor, les demostracions i els rols dels processos fills.
ProcessRunner.java	Construeix les ordres, inicia processos, gestiona els canals, el timeout i la cancel·lació.
ProcessExecutionResult.java	Record immutable amb l'ordre, PID, PPID, codi de sortida, durada, stdout, stderr, timeout i estat.

2.1. Relació entre els fitxers

CreacioProcessos.main()
→ runSupervisor()
→ stdinDemo() / parallelDemo() / timeoutDemo() / pipeDemo()
→ ProcessRunner.javaCommand() i supervise()/startProcess()
→ Processos fills: ECHO, WORKER, PRODUCER i CONSUMER
→ ProcessExecutionResult (dades del resultat)

Quan ProcessBuilder.start() executa una ordre Java, el sistema operatiu crea un PROCÉS FILL amb PID propi. Els fils que llegeixen stdout i stderr pertanyen al procés supervisor; no són processos nous.

2.2. Decisions de disseny

S'utilitza una mateixa classe principal amb diferents rols en lloc de programes separats. ProcessRunner concentra la creació i supervisió dels processos, mentre que ProcessExecutionResult conserva els resultats de manera immutable amb List.copyOf().

Les comandes es passen com a llista d'arguments a ProcessBuilder. La sortida normal (stdout) es manté separada dels diagnòstics (stderr), dos fils lectors consumeixen els fluxos mentre el fill treballa per evitar omplir buffers.

Límits i neteja: waitFor(limit) controla la durada; stopTree() intenta destroy() i després destroyForcibly(). Un shutdown hook gestiona el tancament del supervisor i consulta el conjunt de fills actius.

3. Funcionament general, des de l'inici fins al final

3.1. main(): seleccionar el rol

```
public static void main(String[] args) throws Exception {
  String rol = args.length == 0 ? "supervisor" : args[0].toLowerCase(Locale.ROOT);
  switch (rol) {
     case "supervisor" -> runSupervisor();
     case "echo" -> runEcho();
     case "worker" -> runWorker(args);
     case "producer" -> runProducer();
     case "consumer" -> runConsumer();
     case "--hold-test" -> runHoldTest();
     default -> throw new IllegalArgumentException("Rol desconegut: " + rol);
  }
}
```

Quan s'executa sense arguments, main() inicia el rol supervisor. Quan ProcessBuilder crea un procés fill executant Java amb l'argument "worker", el main() d'aquell procés tria runWorker(). És la mateixa classe Java executada en un altre procés del sistema operatiu, amb PID propi.

3.2. Crear una ordre sense shell

```
public static List<String> javaCommand(String... args) {
  String java = Path.of(System.getProperty("java.home"), "bin", "java").toString();
  List<String> ordre = new ArrayList<>(List.of(
        java, "-cp", System.getProperty("java.class.path"), "CreacioProcessos"));
  Collections.addAll(ordre, args);
  return ordre;
}
```

java.home indica la ubicació del Java que utilitza el procés actual. java.class.path indica on es troben les classes compilades. Els arguments es passen com a elements d'una llista, per exemple [java, -cp, target/classes, CreacioProcessos, worker, ALPHA]. Quan s'executa aquesta ordre, es crea un procés fill que executa Java.

3.3. Iniciar i identificar el procés fill

```
/**
* Inicia un procés i el registra perquè es pugui cancel·lar durant el tancament.
*
* @param ordre executable i arguments del procés
* @return procés iniciat
* @throws IOException si no es pot iniciar el procés o el supervisor ja es tanca
*/
public static Process startProcess(List<String> ordre) throws IOException {
  synchronized (actius) {
    if (tancant) throw new IOException("El supervisor s'està tancant");
    Process p = new ProcessBuilder(ordre).start();
    actius.add(p);
    return p;
  }
}
```

start() crea el procés del sistema operatiu. El supervisor en conserva la referència amb un objecte Process, així pot consultar-ne el PID, gestionar els canals i aturar-lo.

3.4. Capturar stdout i stderr al mateix temps

```
Thread out = drain(p.getInputStream(), stdout, "stdout");
Thread err = drain(p.getErrorStream(), stderr, "stderr");
```

El supervisor inicia un fil per a stdout i un altre per a stderr. drain() llegeix línies i les desa amb data/hora i origen; així es redueix el risc de bloqueig perquè els dos buffers es consumeixen mentre el fill treballa.

3.5. Enviar dades per stdin i indicar EOF

```
// getOutputStream() és l'stdin del fill. En tancar-lo, enviem EOF.
try (OutputStream stdin = p.getOutputStream()) {
  if (entrada != null) stdin.write(entrada.getBytes(StandardCharsets.UTF_8));
}
```

getOutputStream() és la manera que té el pare d'escriure al stdin del fill. Quan es tanca aquest canal, el fill pot detectar EOF (end of file) amb readLine() == null i acabar la lectura.

3.6. Esperar, comprovar timeout i recuperar el resultat

```
// El tancament de stdin permet al fill detectar el final de l'entrada (EOF).
boolean timeout = !p.waitFor(limit.toMillis(), TimeUnit.MILLISECONDS);
if (timeout) stopTree(p, "timeout de " + rol);
out.join(); // Espera el fil que llegeix stdout.
err.join(); // Espera el fil que llegeix stderr.
return resultat(p, ordre, rol, inici, stdout, stderr, timeout);
```

waitFor() espera el procés; join() espera els fils lectors. Si el procés supera el límit, stopTree() intenta acabar-lo ordenadament i després forçadament. Finalment, ProcessExecutionResult conserva les dades de l'execució.

4. Proves realitzades i resultats obtinguts

4.1. Echo: stdin, stdout, stderr i EOF (End of file)

El supervisor envia "Ebre\nprocessos\nFI\n" a un fill ECHO. El fill transforma les dues primeres línies a majúscules i s'atura amb FI o EOF.

```
ECHO pid=13573 ppid=13552 exit=0 temps=154ms timeout=false estat=OK ordre=[/home/alumnat/.jdks/openjdk-25.0.2/bin/java, -cp, target/classes, CreacioProcessos, echo]
18:02:26.882 [stdout] EBRE
18:02:26.891 [stdout] PROCESSOS
18:02:26.882 [stderr] [ECHO pid=13573 ppid=13552 thread=main] inici
18:02:26.891 [stderr] [ECHO pid=13573 ppid=13552 thread=main] final
```

COM PROVAR-HO:

Execució del rol Echo: printf 'Ebre\nprocessos\nFI\n' | java -cp target/classes CreacioProcessos echo

```
alumnat@insebre:~/Escriptori/2NDAM/ProgramacióDeProcessos/ProgramacioProcessos_pt1$ printf 'Ebre\nprocessos\nFI\n' | java -cp target/classes CreacioProcessos echo
[ECHO pid=18567 ppid=12746 thread=main] inici
EBRE
PROCESSOS
[ECHO pid=18567 ppid=12746 thread=main] final
```

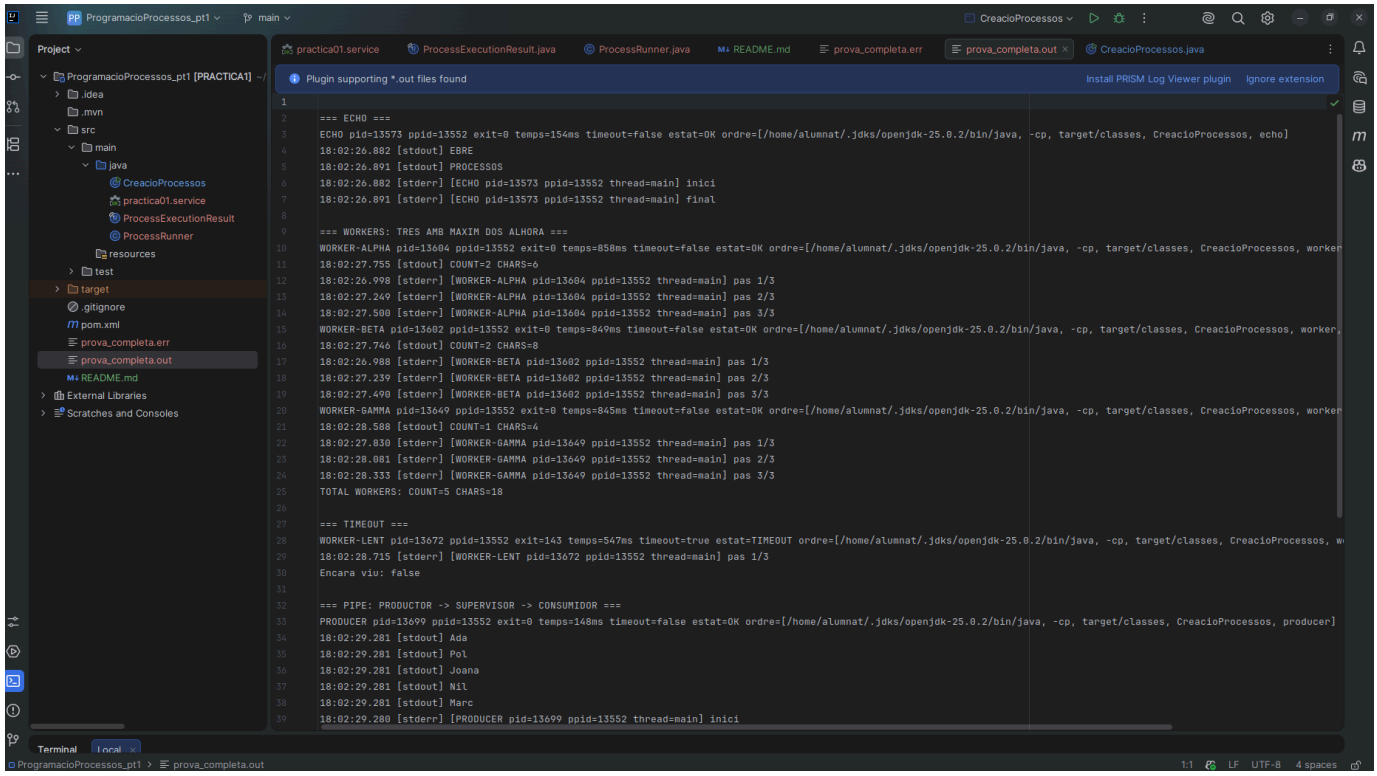
Es veu la sortida de EBRE i PROCESSOS, i la finalització quan acaba de llegir l'execució, que es quan arriba el EOF o el fi.

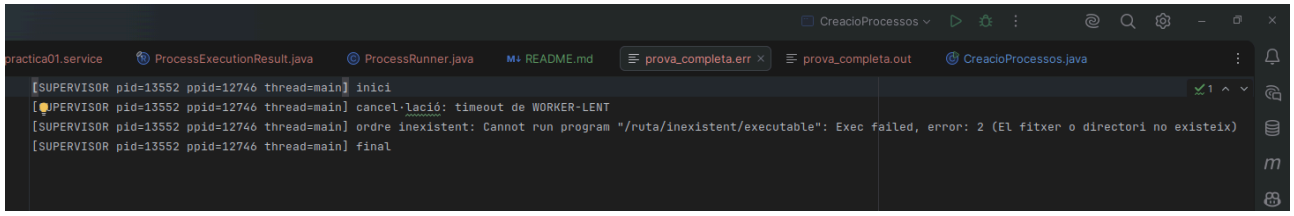
4.2. Workers: tres processos i màxim dos simultanis

```java
/** Executa tres workers en paral·lel, amb un màxim de dos alhora, i suma els resultats. */
private static void parallelDemo() throws Exception {
  System.out.println("\n=== WORKERS: TRES AMB MAXIM DOS ALHORA ===");
  String[] noms = {"ALPHA", "BETA", "GAMMA"};
  String[] entrades = {"Ada\nPol\n", "Joana\nNil\n", "Marc\n"};
  try (var pool = Executors.newFixedThreadPool(2)) {
    var tasques = new ArrayList<Future<ProcessExecutionResult>>();
    for (int i = 0; i < noms.length; i++) {
      String nom = noms[i], entrada = entrades[i];
      // Cada tasca crea un PROCÉS worker i espera que acabi.
      tasques.add(pool.submit(() -> ProcessRunner.supervise(
          ProcessRunner.javaCommand("worker", nom),
          "WORKER-" + nom, Duration.ofSeconds(5), entrada)));
    }
```

Comanda per executar des del terminal i guardar l'execució completa a un arxiu (prova_completa.out) i els errors (prova_completa.err). Pd: tot això es fa des del directori del projecte.

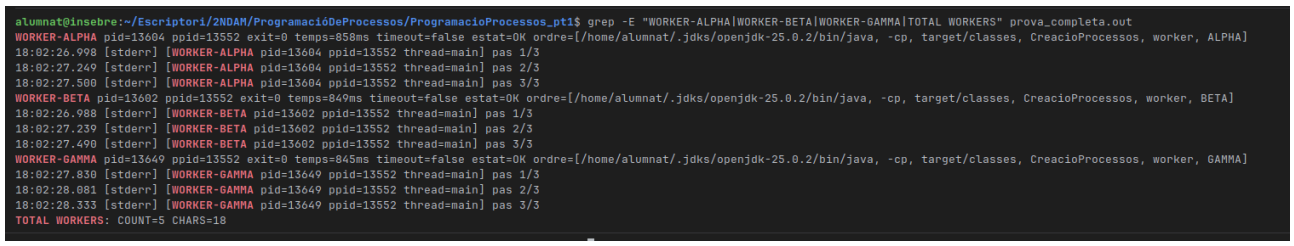
1. java -cp target/classes CreacioProcessos > prova_completa.out 2> prova_completa.err





2. grep -E "WORKER-ALPHA|WORKER-BETA|WORKER-GAMMA|TOTAL WORKERS" prova_completa.out

→ Agrupo l'output del arxiu creat amb els workers



Els missatges: pas 1/3, pas 2/3 i pas 3/3 són diagnòstics enviats per stderr, no errors. El canal stderr també pot contenir errors reals, per això cal interpretar el missatge i el codi de sortida.

Tasques: ALPHA, BETA, GAMMA (Subprocessos -PID, del pare - PPID)

Worker	PID	PPID	Exit	Durada	Resultat
ALPHA	13604	13552	0	858 ms	COUNT=2 CHARS=6
BETA	13602	13552	0	849 ms	COUNT=2 CHARS=8
GAMMA	13649	13552	0	845 ms	COUNT=1 CHARS=4

Els resultats mostren que el supervisor ha executat tres processos workers independents, cadascun amb un PID propi i amb el mateix PPID (13552).

Mitjançant Executors.newFixedThreadPool(2), s'ha limitat l'execució a un màxim de dues tasques de supervisió simultànies. Inicialment, els dos fils del pool supervisen ALPHA i BETA, mentre que GAMMA queda a la espera que un dels fils de la pool quede lliure.

Cada tasca utilitza waitFor() per esperar que finalitzi el procés fill corresponent. Quan ALPHA o BETA acaba i finalitza la seva tasca de supervisió, el fil queda disponible i pot començar GAMMA.

Aquesta execució permet comprovar la concurrència dels processos ALPHA i BETA

4.2.1. Mapa inicial dels processos (taula obligatòria)

Prova amb PID diferents a les prèvies, ja que cada vegada que executo els ids canvïen

Mitjançant la següent comanda gràcies a l'ajuda de la IA, s'extreu el pstree:

Iniciem el supervisor en segon pla

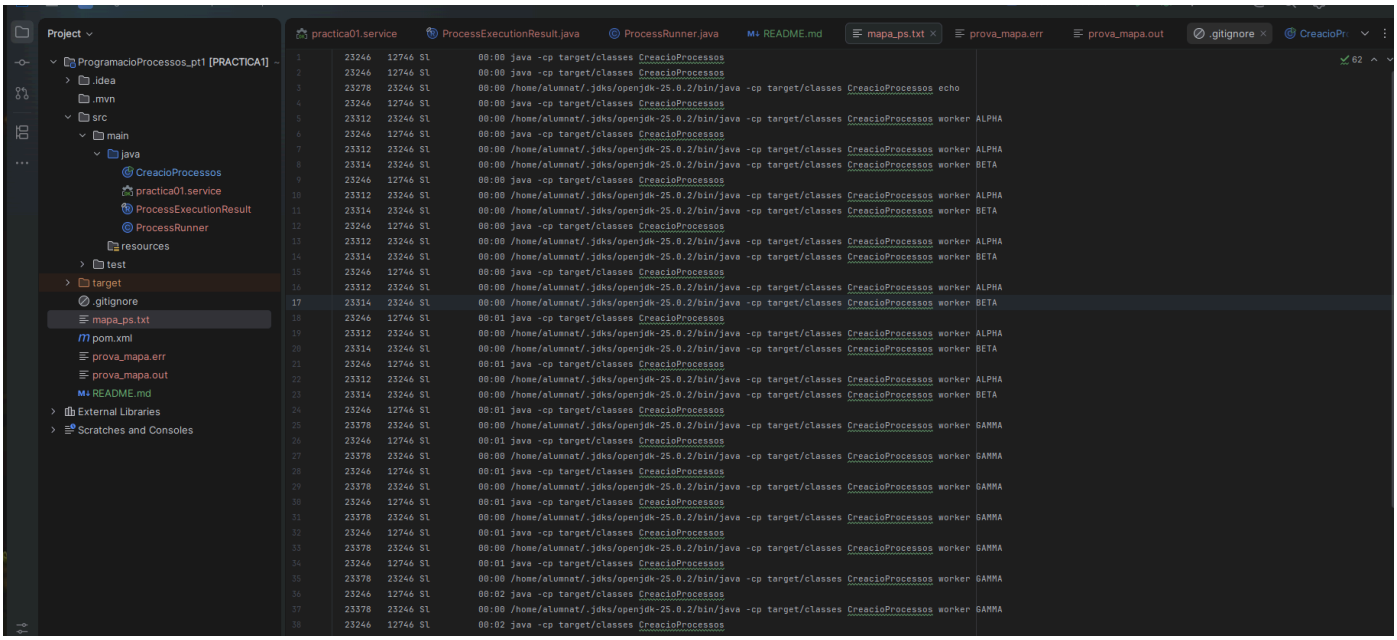
java -cp target/classes CreacioProcessos > prova_mapa.out 2> prova_mapa.err &

Guardem el PID del supervisor

PID=$!

echo "PID DEL SUPERVISOR: PID"#Capturemelsprocessoscada0,1segonswhilekill − 0"**PID" 2>/dev/null; do**

ps -eo pid,ppid,stat,etime,cmd | awk -v pare="PID"'**1==pare ||** $2 == pare'sleep0.1done|teemapa_p s.txtwait"**PID"**



Rol	PID	PPID	Estat observat (ps)	Codi retorn
Supervisor	23246	12746	sl	0 (execució global)
Worker ALPHA	23312	23246	sl	0
Worker BETA	23314	23246	sl	0
Worker GAMMA	23378	23246	sl	0

Què significa Sl?

- S: el procés està en espera interrompible, per exemple durant un sleep() o esperant dades.
- l: el procés té múltiples fils (multithreaded).

4.3. Timeout i finalització

```
=== TIMEOUT ===
WORKER-LENT pid=23424 ppid=23246 exit=143 temps=537ms timeout=true estat=TIMEOUT ordre=[/home/alumnat/.jdks/openjdk-25.0.2/bin/java, -cp, target/classes, CreacioProcessos, worker, LENT, --sleep, 2000]
19:12:32.083 [stderr] [WORKER-LENT pid=23424 ppid=23246 thread=main] pas 1/3
Encara viu: false
```

La prova assigna 500 ms de límit a un worker que espera més temps. El supervisor registra la cancel·lació, finalitza el procés i comprova que ja no viu “apareix false”.

COM PROVAR-HO:

1. Execució del supervisor mitjançant les següents comandes d’agrupació:

grep -E "WORKER-LENT|TIMEOUT|Encara viu" prova_mapa.out

```
alumnat@insebre:~/Escriptori/2NDAM/ProgramacioDeProcessos/ProgramacioProcessos_pt1$ grep -E "WORKER-LENT|TIMEOUT|Encara viu" prova_completa.out
grep: prova_completa.out: El fitxer o directori no existeix

alumnat@insebre:~/Escriptori/2NDAM/ProgramacióDeProcessos/ProgramacioProcessos_pt1$ grep -E "WORKER-LENT|TIMEOUT|Encara viu" prova_mapa.out
=== TIMEOUT ===
WORKER-LENT pid=23424 ppid=23246 exit=143 temps=537ms timeout=true estat=TIMEOUT ordre=[/home/alumnat/.jdks/openjdk-25.0.2/bin/java, -cp, target/classes, CreacioProcessos, worker, LENT, --sleep, 2
000]
19:12:32.083 [stderr] [WORKER-LENT pid=23424 ppid=23246 thread=main] pas 1/3
Encara viu: false
```

grep -i "cancel" prova_mapa.err

```
alumnat@insebre:~/Escriptori/2NDAM/ProgramacióDeProcessos/ProgramacioProcessos_pt1$ grep -i "cancel" prova_mapa.err
[SUPERVISOR pid=23246 ppid=12746 thread=main] cancel·lació: timeout de WORKER-LENT
```

4.4. Pipe: PRODUCER → SUPERVISOR → CONSUMER

```
=== PIPE: PRODUCTOR -> SUPERVISOR -> CONSUMIDOR ===
PRODUCER pid=23463 ppid=23246 exit=0 temps=122ms timeout=false estat=OK ordre=[/home/alumnat/.jdks/openjdk-25.0.2/bin/java, -cp, target/classes, CreacioProcessos, producer]
19:12:32.630 [stdout] Ada
19:12:32.630 [stdout] Pol
19:12:32.630 [stdout] Joana
19:12:32.631 [stdout] Nil
19:12:32.631 [stdout] Marc
19:12:32.630 [stderr] [PRODUCER pid=23463 ppid=23246 thread=main] inici
19:12:32.631 [stderr] [PRODUCER pid=23463 ppid=23246 thread=main] final
CONSUMER pid=23462 ppid=23246 exit=0 temps=124ms timeout=false estat=OK ordre=[/home/alumnat/.jdks/openjdk-25.0.2/bin/java, -cp, target/classes, CreacioProcessos, consumer]
19:12:32.644 [stdout] COUNT=5 CHARS=18
19:12:32.632 [stderr] [CONSUMER pid=23462 ppid=23246 thread=main] inici
19:12:32.645 [stderr] [CONSUMER pid=23462 ppid=23246 thread=main] final
```

grep -E "PIPE|PRODUCER|CONSUMER|COUNT=5 CHARS=18" prova_mapa.out

```
alumnat@insebre:~/Escriptori/2NDAM/ProgramacióDeProcessos/ProgramacioProcessos_pt1$ grep -E "PIPE|PRODUCER|CONSUMER|COUNT=5 CHARS=18" prova_mapa.out
TOTAL WORKERS: COUNT=5 CHARS=18
=== PIPE: PRODUCTOR -> SUPERVISOR -> CONSUMIDOR ===
PRODUCER pid=23463 ppid=23246 exit=0 temps=122ms timeout=false estat=OK ordre=[/home/alumnat/.jdks/openjdk-25.0.2/bin/java, -cp, target/classes, CreacioProcessos, producer]
19:12:32.630 [stderr] [PRODUCER pid=23463 ppid=23246 thread=main] inici
19:12:32.631 [stderr] [PRODUCER pid=23463 ppid=23246 thread=main] final
CONSUMER pid=23462 ppid=23246 exit=0 temps=124ms timeout=false estat=OK ordre=[/home/alumnat/.jdks/openjdk-25.0.2/bin/java, -cp, target/classes, CreacioProcessos, consumer]
19:12:32.644 [stdout] COUNT=5 CHARS=18
19:12:32.632 [stderr] [CONSUMER pid=23462 ppid=23246 thread=main] inici
19:12:32.645 [stderr] [CONSUMER pid=23462 ppid=23246 thread=main] final
```

El producer genera els 5 noms escrivint-ho al stdout. El supervisor rep les linies i mitjançant un fil pont, ho envia al Consumer. Aquest retorna una comptabilització de les línies i els caracters i s'arriba al EOF.

4.5. Error de sortida 7 i ordre inexistent

```
=== CODI 7 ===
WORKER-ERROR pid=23511 ppid=23246 exit=7 temps=868ms timeout=false estat=ERROR ordre=[/home/alumnat/.jdks/openjdk-25.0.2/bin/java, -cp, target/classes, CreacioProcessos, worker, ERROR, --exit-code, 7]
```

```
19:12:33.511 [stdout] COUNT=0 CHARS=0
19:12:32.751 [stderr] [WORKER-ERROR pid=23511 ppid=23246 thread=main] pas 1/3
19:12:33.003 [stderr] [WORKER-ERROR pid=23511 ppid=23246 thread=main] pas 2/3
19:12:33.255 [stderr] [WORKER-ERROR pid=23511 ppid=23246 thread=main] pas 3/3
```

Això demostra que un procés fill pot fallar sense que el supervisor deixi de recollir i registrar els altres resultats. L'executable inexistent provoca IOException en intentar fer start().

PROVA:

grep -i "ordre inexistent" prova_completa.err

```java
/** Comprova que un executable inexistent es comunica com un error d'entrada/sortida. */
private static void missingCommandDemo() throws Exception {
  System.out.println("\n=== ORDRE INEXISTENT ===");
  try {
     ProcessRunner.supervise(List.of("/ruta/inexistent/executable"),
           "MISSING", Duration.ofSeconds(2), null);
  } catch (IOException e) {
     diagnostic("SUPERVISOR", "ordre inexistent: " + e.getMessage());
  }
}
```

```
alumnat@insebre:~/Escriptori/2NDAM/ProgramacióDeProcessos/ProgramacioProcessos_pt1$ grep -i "ordre inexistent" prova_mapa.err
[SUPERVISOR pid=23246 ppid=12746 thread=main] ordre inexistent: Cannot run program "/ruta/inexistent/executable": Exec failed, error: 2 (El fitxer o directori no existeix)
```

4.6. testos unitaris

Nom del test	**Què comprova?**
testEcho()	Rep dades i les retorna en majúscules.
testStderr()	Captura els missatges de diagnòstic.
testWorkers()	Executa tres workers i comprova el recompte total.
testLimitConcurrencia()	Permet un màxim de dos workers simultanis.
testTimeout()	Atura un procés quan supera el temps límit.
testExitCode7()	Detecta un procés que acaba amb codi d'error 7.
testOrdreInexistent()	Detecta una ordre inexistent mitjançant IOException.

```
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 9.842 s -- in ProcessRunnerTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  12.542 s
[INFO] Finished at: 2026-10-26T20:12:22+01:00
[INFO] ------------------------------------------------------------------------
alumnat@insebre:~/Escriptori/2NDAM/ProgramacióDeProcessos/ProgramacioProcessos_pt1$
```

5. Integració com a servei systemd

5.1. Configurar practica01.service

A diferència de l'enunciat de la pràctica, el systemd l'he configurat amb java25, ja que es el JDK que tenía instal·lat

practica01.service | ProcessExecutionResult.java | ProcessRunner.java | README.md | mapa_ps.txt | prova_mapa.err | prova_mapa.out | .gitignore

```
[Unit]
Description=PAC1

[Service]
Type=oneshot
User=alumnat
WorkingDirectory=/home/alumnat/Escriptori/2NDAM/ProgramacióDeProcessos/ProgramacioProcessos_pt1
ExecStart=/home/alumnat/.jdks/openjdk-25.0.2/bin/java -jar /home/alumnat/Escriptori/2NDAM/ProgramacióDeProcessos/ProgramacioProcessos_pt1/supervisor.jar
TimeoutStartSec=90s
TimeoutStopSec=5s
KillMode=control-group
StandardOutput=journal
StandardError=journal
```

Commandes a provar:

sudo cp practica01.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl start practica01.service
sudo systemctl status practica01.service
sudo journalctl -u practica01.service -b --no-pager

```
alumnat@insebre:~/Escriptori/2NDAM/ProgramacióDeProcessos/ProgramacioProcessos_pt1$ sudo cp practica01.service /etc/systemd/system/
alumnat@insebre:~/Escriptori/2NDAM/ProgramacióDeProcessos/ProgramacioProcessos_pt1$ sudo systemctl daemon-reload
alumnat@insebre:~/Escriptori/2NDAM/ProgramacióDeProcessos/ProgramacioProcessos_pt1$ sudo systemctl start practica01.service
alumnat@insebre:~/Escriptori/2NDAM/ProgramacióDeProcessos/ProgramacioProcessos_pt1$ sudo systemctl status practica01.service
○ practica01.service - PAC1
     Loaded: loaded (/etc/systemd/system/practica01.service; static)
     Active: inactive (dead)

d'oct. 26 21:19:55 insebre java[40066]: 21:19:55.067 [stdout] COUNT=0 CHARS=0
d'oct. 26 21:19:55 insebre java[40066]: 21:19:54.307 [stderr] [WORKER-ERROR pid=40259 ppid=40066 thread=main] pas 1/3
d'oct. 26 21:19:55 insebre java[40066]: 21:19:54.559 [stderr] [WORKER-ERROR pid=40259 ppid=40066 thread=main] pas 2/3
d'oct. 26 21:19:55 insebre java[40066]: 21:19:54.810 [stderr] [WORKER-ERROR pid=40259 ppid=40066 thread=main] pas 3/3
d'oct. 26 21:19:55 insebre java[40066]: === ORDRE INEXISTENT ===
d'oct. 26 21:19:55 insebre java[40066]: [SUPERVISOR pid=40066 ppid=1 thread=main] ordre inexistent: Cannot run program "/ruta/inexistent/executable": Exec failed, error: 2 (El fitxer o director
d'oct. 26 21:19:55 insebre java[40066]: [SUPERVISOR pid=40066 ppid=1 thread=main] final
d'oct. 26 21:19:55 insebre systemd[1]: practica01.service: Deactivated successfully.
d'oct. 26 21:19:55 insebre systemd[1]: Finished practica01.service - PAC1.
```

```
alumnat@insebre:~/Escriptori/2NDAM/ProgramacióDeProcessos/ProgramacioProcessos_pt1$ sudo journalctl -u practica01.service -b --no-pager
d'oct. 26 21:19:51 insebre systemd[1]: Starting practica01.service - PAC1...
d'oct. 26 21:19:51 insebre java[40066]: [SUPERVISOR pid=40066 ppid=1 thread=main] inici
d'oct. 26 21:19:51 insebre java[40066]: === ECHO ===
d'oct. 26 21:19:51 insebre java[40066]: ECHO pid=40087 ppid=40066 exit=0 temps=153ms timeout=false estat=OK ordre=[/home/alumnat/.jdks/openjdk-25.0.2/bin/java, -cp, /home/alumnat/Escriptori/2NDAM/
ProgramacióDeProcessos/ProgramacioProcessos_pt1/supervisor.jar, CreacioProcessos, echo]
d'oct. 26 21:19:51 insebre java[40066]: 21:19:51.654 [stdout] EBRE
d'oct. 26 21:19:51 insebre java[40066]: 21:19:51.667 [stdout] PROCESSOS
d'oct. 26 21:19:51 insebre java[40066]: 21:19:51.654 [stderr] [ECHO pid=40087 ppid=40066 thread=main] inici
d'oct. 26 21:19:51 insebre java[40066]: 21:19:51.667 [stderr] [ECHO pid=40087 ppid=40066 thread=main] final
d'oct. 26 21:19:51 insebre java[40066]: === WORKERS: TRES AMB MAXIM DOS ALHORA ===
d'oct. 26 21:19:52 insebre java[40066]: WORKER-ALPHA pid=40114 ppid=40066 exit=0 temps=884ms timeout=false estat=OK ordre=[/home/alumnat/.jdks/openjdk-25.0.2/bin/java, -cp, /home/alumnat/Escriptor
i/2NDAM/ProgramacióDeProcessos/ProgramacioProcessos_pt1/supervisor.jar, CreacioProcessos, worker, ALPHA]
d'oct. 26 21:19:52 insebre java[40066]: 21:19:52.561 [stdout] COUNT=2 CHARS=6
d'oct. 26 21:19:52 insebre java[40066]: 21:19:51.802 [stderr] [WORKER-ALPHA pid=40114 ppid=40066 thread=main] pas 1/3
d'oct. 26 21:19:52 insebre java[40066]: 21:19:52.053 [stderr] [WORKER-ALPHA pid=40114 ppid=40066 thread=main] pas 2/3
d'oct. 26 21:19:52 insebre java[40066]: 21:19:52.305 [stderr] [WORKER-ALPHA pid=40114 ppid=40066 thread=main] pas 3/3
d'oct. 26 21:19:52 insebre java[40066]: WORKER-BETA pid=40116 ppid=40066 exit=0 temps=879ms timeout=false estat=OK ordre=[/home/alumnat/.jdks/openjdk-25.0.2/bin/java, -cp, /home/alumnat/Escriptori
/2NDAM/ProgramacióDeProcessos/ProgramacioProcessos_pt1/supervisor.jar, CreacioProcessos, worker, BETA]
d'oct. 26 21:19:52 insebre java[40066]: 21:19:52.556 [stdout] COUNT=2 CHARS=8
d'oct. 26 21:19:52 insebre java[40066]: 21:19:51.797 [stderr] [WORKER-BETA pid=40116 ppid=40066 thread=main] pas 1/3
d'oct. 26 21:19:52 insebre java[40066]: 21:19:52.048 [stderr] [WORKER-BETA pid=40116 ppid=40066 thread=main] pas 2/3
d'oct. 26 21:19:52 insebre java[40066]: 21:19:52.299 [stderr] [WORKER-BETA pid=40116 ppid=40066 thread=main] pas 3/3
d'oct. 26 21:19:53 insebre java[40066]: WORKER-GAMMA pid=40163 ppid=40066 exit=0 temps=888ms timeout=false estat=OK ordre=[/home/alumnat/.jdks/openjdk-25.0.2/bin/java, -cp, /home/alumnat/Escriptor
i/2NDAM/ProgramacióDeProcessos/ProgramacioProcessos_pt1/supervisor.jar, CreacioProcessos, worker, GAMMA]
d'oct. 26 21:19:53 insebre java[40066]: 21:19:53.443 [stdout] COUNT=1 CHARS=4
d'oct. 26 21:19:53 insebre java[40066]: 21:19:52.684 [stderr] [WORKER-GAMMA pid=40163 ppid=40066 thread=main] pas 1/3
d'oct. 26 21:19:53 insebre java[40066]: 21:19:52.936 [stderr] [WORKER-GAMMA pid=40163 ppid=40066 thread=main] pas 2/3
d'oct. 26 21:19:53 insebre java[40066]: 21:19:53.187 [stderr] [WORKER-GAMMA pid=40163 ppid=40066 thread=main] pas 3/3
d'oct. 26 21:19:53 insebre java[40066]: TOTAL WORKERS: COUNT=5 CHARS=18
d'oct. 26 21:19:53 insebre java[40066]: === TIMEOUT ===
d'oct. 26 21:19:53 insebre java[40066]: [SUPERVISOR pid=40066 ppid=1 thread=main] cancel·lació: timeout de WORKER-LENT
d'oct. 26 21:19:54 insebre java[40066]: WORKER-LENT pid=40188 ppid=40066 exit=143 temps=551ms timeout=true estat=TIMEOUT ordre=[/home/alumnat/.jdks/openjdk-25.0.2/bin/java, -cp, /home/alumnat/Escr
iptori/2NDAM/ProgramacióDeProcessos/ProgramacioProcessos_pt1/supervisor.jar, CreacioProcessos, worker, LENT, --sleep, 2000]
d'oct. 26 21:19:54 insebre java[40066]: 21:19:53.580 [stderr] [WORKER-LENT pid=40188 ppid=40066 thread=main] pas 1/3
d'oct. 26 21:19:54 insebre java[40066]: Encara viu: false
d'oct. 26 21:19:54 insebre java[40066]: === PIPE: PRODUCTOR -> SUPERVISOR -> CONSUMIDOR ===
d'oct. 26 21:19:54 insebre java[40066]: PRODUCER pid=40214 ppid=40066 exit=0 temps=170ms timeout=false estat=OK ordre=[/home/alumnat/.jdks/openjdk-25.0.2/bin/java, -cp, /home/alumnat/Escriptori/2N
DAM/ProgramacióDeProcessos/ProgramacioProcessos_pt1/supervisor.jar, CreacioProcessos, producer]
d'oct. 26 21:19:54 insebre java[40066]: 21:19:54.156 [stdout] Ada
d'oct. 26 21:19:54 insebre java[40066]: 21:19:54.156 [stdout] Pol
d'oct. 26 21:19:54 insebre java[40066]: 21:19:54.156 [stdout] Joana
d'oct. 26 21:19:54 insebre java[40066]: 21:19:54.156 [stdout] Nil
```

```
d'oct. 26 21:19:54 insebre java[40066]: 21:19:54.157 [stdout] Marc
d'oct. 26 21:19:54 insebre java[40066]: 21:19:54.156 [stderr] [PRODUCER pid=40214 ppid=40066 thread=main] inici
d'oct. 26 21:19:54 insebre java[40066]: 21:19:54.157 [stderr] [PRODUCER pid=40214 ppid=40066 thread=main] final
d'oct. 26 21:19:54 insebre java[40066]: CONSUMER pid=40213 ppid=40066 exit=0 temps=173ms timeout=false estat=OK ordre=[/home/alumnat/.jdks/openjdk-25.0.2/bin/java, -cp, /home/alumnat/Escriptori/2N
DAM/ProgramacióDeProcessos/ProgramacioProcessos_pt1/supervisor.jar, CreacioProcessos, consumer]
d'oct. 26 21:19:54 insebre java[40066]: 21:19:54.177 [stdout] COUNT=5 CHARS=18
d'oct. 26 21:19:54 insebre java[40066]: 21:19:54.160 [stderr] [CONSUMER pid=40213 ppid=40066 thread=main] inici
d'oct. 26 21:19:54 insebre java[40066]: 21:19:54.177 [stderr] [CONSUMER pid=40213 ppid=40066 thread=main] final
d'oct. 26 21:19:54 insebre java[40066]: === CODI 7 ===
d'oct. 26 21:19:55 insebre java[40066]: WORKER-ERROR pid=40259 ppid=40066 exit=7 temps=891ms timeout=false estat=ERROR ordre=[/home/alumnat/.jdks/openjdk-25.0.2/bin/java, -cp, /home/alumnat/Escrip
tori/2NDAM/ProgramacióDeProcessos/ProgramacioProcessos_pt1/supervisor.jar, CreacioProcessos, worker, ERROR, --exit-code, 7]
d'oct. 26 21:19:55 insebre java[40066]: 21:19:55.067 [stdout] COUNT=0 CHARS=0
d'oct. 26 21:19:55 insebre java[40066]: 21:19:54.307 [stderr] [WORKER-ERROR pid=40259 ppid=40066 thread=main] pas 1/3
d'oct. 26 21:19:55 insebre java[40066]: 21:19:54.559 [stderr] [WORKER-ERROR pid=40259 ppid=40066 thread=main] pas 2/3
d'oct. 26 21:19:55 insebre java[40066]: 21:19:54.810 [stderr] [WORKER-ERROR pid=40259 ppid=40066 thread=main] pas 3/3
d'oct. 26 21:19:55 insebre java[40066]: === ORDRE INEXISTENT ===
d'oct. 26 21:19:55 insebre java[40066]: [SUPERVISOR pid=40066 ppid=1 thread=main] ordre inexistent: Cannot run program "/ruta/inexistent/executable": Exec failed, error: 2 (El fitxer o directori n
o existeix)
d'oct. 26 21:19:55 insebre java[40066]: [SUPERVISOR pid=40066 ppid=1 thread=main] final
d'oct. 26 21:19:55 insebre systemd[1]: practica01.service: Deactivated successfully.
d'oct. 26 21:19:55 insebre systemd[1]: Finished practica01.service - PAC1.
d'oct. 26 21:19:55 insebre systemd[1]: practica01.service: Consumed 2.298s CPU time.
alumnat@insebre:~/Escriptori/2NDAM/ProgramacióDeProcessos/ProgramacioProcessos_pt1$
```

5.2. Prova guiada de parada

Mitjançant aquest mètode:

```
            case "--hold-test" -> runHoldTest();
            default -> throw new IllegalArgumentException("Rol desconegut: " + rol);
        }
    }
```

```
    return new int[]{count, chars};
}

/** Manté un worker actiu durant prou temps per provar l'aturada del servei. */
private static void runHoldTest() throws Exception {  1 usage  new *
    diagnostic( rol: "SUPERVISOR",  missatge: "prova de servei llarga");
    ProcessRunner.supervise(ProcessRunner.javaCommand( ...args: "worker", "SERVICE", "--sleep", "60000"),
            rol: "WORKER-SERVICE", Duration.ofMinutes(4),  entrada: null);
}
```

Al agregar al systemd: “--hold-test, s’executarà la prova”

```
ExecStart=/home/alumnat/.jdks/openjdk-25.0.2/bin/java -jar /home/alumnat/Escriptori/2NDAM/ProgramacióDeProcessos/ProgramacioProcessos_pt1/supervisor.jar --hold-test
```

S’actualitza el servei i gràcies al holdtest fa que estigui actiu per més temps:

```
alumnat@insebre:~/Escriptori/2NDAM/ProgramacióDeProcessos/ProgramacioProcessos_pt1$ sudo cp practica01.service /etc/systemd/system/
alumnat@insebre:~/Escriptori/2NDAM/ProgramacióDeProcessos/ProgramacioProcessos_pt1$ sudo systemctl daemon-reload
alumnat@insebre:~/Escriptori/2NDAM/ProgramacióDeProcessos/ProgramacioProcessos_pt1$ sudo systemctl start practica01.service
gramacioProcessos_pt1 › practica01.service
```

Obro una segona terminal i amb **sudo systemctl status practica01.service --no-pager -l** i abans que finalitze el temps d'execució, utilitzo la comanda ps amb un filtre grep per identificar els dos processos actius: el supervisor (PID 42789) i el worker SERVICE (PID 42810).

Comprovo que el worker té com a PPID el PID del supervisor (42789), cosa que demostra la relació pare-fill entre els dos processos:

```
gramacioProcessos_pt1$ sudo systemctl status practica01.service --no-pager -l
● practica01.service - PAC1
     Loaded: loaded (/etc/systemd/system/practica01.service; static)
     Active: activating (start) since Mon 2026-10-26 21:33:33 CET; 4s ago
   Main PID: 42789 (java)
      Tasks: 40 (limit: 16061)
     Memory: 51.1M (peak: 52.2M)
        CPU: 797ms
     CGroup: /system.slice/practica01.service
             ├─42789 /home/alumnat/.jdks/openjdk-25.0.2/bin/java -jar /home/alumnat/Escriptori/2NDAM/ProgramacióDeProcessos/ProgramacioProcessos_pt1/supervisor.jar --hold-test
             └─42810 /home/alumnat/.jdks/openjdk-25.0.2/bin/java -cp /home/alumnat/Escriptori/2NDAM/ProgramacióDeProcessos/ProgramacioProcessos_pt1/supervisor.jar CreacioProcessos work
er SERVICE --sleep 60000

d'oct. 26 21:33:33 insebre systemd[1]: Starting practica01.service - PAC1...
d'oct. 26 21:33:34 insebre java[42789]: [SUPERVISOR pid=42789 ppid=1 thread=main] prova de servei llarga
alumnat@insebre:~/Escriptori/2NDAM/ProgramacióDeProcessos/ProgramacioProcessos_pt1$ ps -eo pid,ppid,stat,etime,cmd | grep -E 'supervisor.jar|creacioProcessos'
  42789       1 Ssl        00:11 /home/alumnat/.jdks/openjdk-25.0.2/bin/java -jar /home/alumnat/Escriptori/2NDAM/ProgramacióDeProcessos/ProgramacioProcessos_pt1/supervisor.jar --hold-t
est
  42810   42789 Sl         00:10 /home/alumnat/.jdks/openjdk-25.0.2/bin/java -cp /home/alumnat/Escriptori/2NDAM/ProgramacióDeProcessos/ProgramacioProcessos_pt1/supervisor.jar CreacioPr
ocessos worker SERVICE --sleep 60000
  42855   42104 S+         00:00 grep --color=auto -E supervisor.jar|creacioProcessos
alumnat@insebre:~/Escriptori/2NDAM/ProgramacióDeProcessos/ProgramacioProcessos_pt1$
```

Després fer un reload ficant-ho com l procès systemd normal

```
alumnat@insebre:~/Escriptori/2NDAM/ProgramacióDeProcessos/ProgramacioProcessos_pt1$ sudo systemctl start practica01.service
Job for practica01.service failed because a timeout was exceeded.
See "systemctl status practica01.service" and "journalctl -xeu practica01.service" for details.

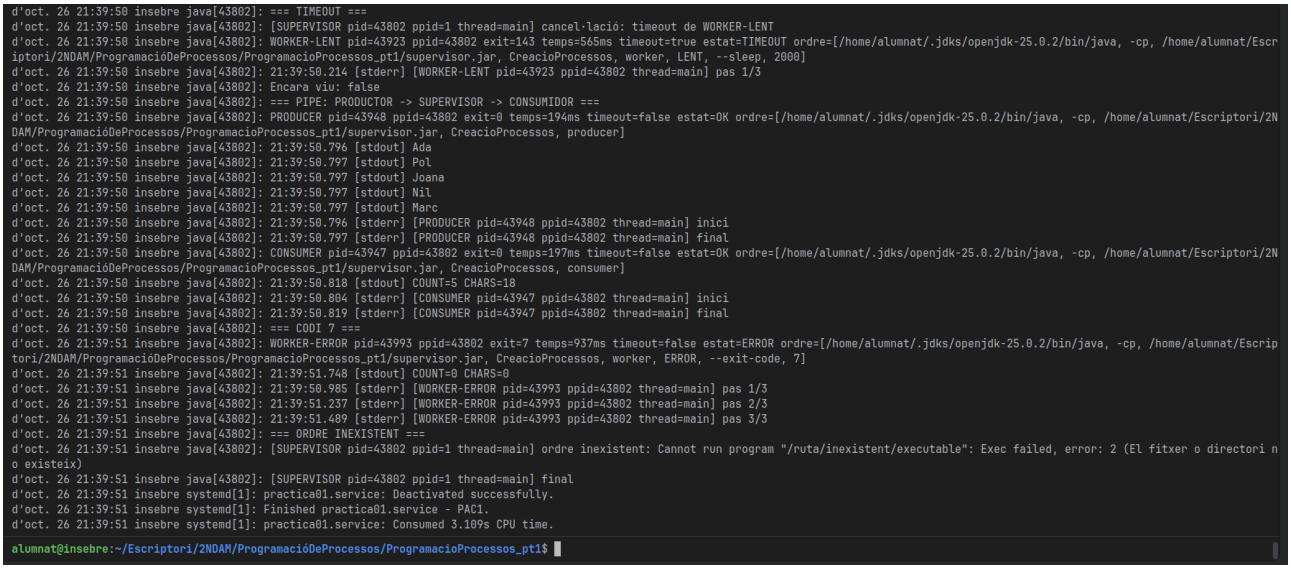
alumnat@insebre:~/Escriptori/2NDAM/ProgramacióDeProcessos/ProgramacioProcessos_pt1$ sudo cp practica01.service /etc/systemd/system/

alumnat@insebre:~/Escriptori/2NDAM/ProgramacióDeProcessos/ProgramacioProcessos_pt1$ sudo systemctl daemon-reload

alumnat@insebre:~/Escriptori/2NDAM/ProgramacióDeProcessos/ProgramacioProcessos_pt1$ sudo systemctl start practica01.service

alumnat@insebre:~/Escriptori/2NDAM/ProgramacióDeProcessos/ProgramacioProcessos_pt1$ sudo journalctl -u practica01.service -b --no-pager
d'oct. 26 21:19:51 insebre systemd[1]: Starting practica01.service - PAC1...
d'oct. 26 21:19:51 insebre java[40066]: [SUPERVISOR pid=40066 ppid=1 thread=main] inici
d'oct. 26 21:19:51 insebre java[40066]: === ECHO ===
d'oct. 26 21:19:51 insebre java[40066]: ECHO pid=40087 ppid=40066 exit=0 temps=153ms timeout=false estat=OK ordre=[/home/alumnat/.jdks/openjdk-25.0.2/bin/java, -cp, /home/alumnat/Escriptori/2NDAM/
ProgramacióDeProcessos/ProgramacioProcessos_pt1/supervisor.jar, CreacioProcessos, echo]
d'oct. 26 21:19:51 insebre java[40066]: 21:19:51.654 [stdout] EBRE
d'oct. 26 21:19:51 insebre java[40066]: 21:19:51.667 [stdout] PROCESSOS
d'oct. 26 21:19:51 insebre java[40066]: 21:19:51.654 [stderr] [ECHO pid=40087 ppid=40066 thread=main] inici
d'oct. 26 21:19:51 insebre java[40066]: 21:19:51.667 [stderr] [ECHO pid=40087 ppid=40066 thread=main] final
d'oct. 26 21:19:51 insebre java[40066]: === WORKERS: TRES AMB MAXIM DOS ALHORA ===
d'oct. 26 21:19:52 insebre java[40066]: WORKER-ALPHA pid=40114 ppid=40066 exit=0 temps=884ms timeout=false estat=OK ordre=[/home/alumnat/.jdks/openjdk-25.0.2/bin/java, -cp, /home/alumnat/Escriptor
i/2NDAM/ProgramacióDeProcessos/ProgramacioProcessos_pt1/supervisor.jar, CreacioProcessos, worker, ALPHA]
d'oct. 26 21:19:52 insebre java[40066]: 21:19:52.561 [stdout] COUNT=2 CHARS=6
d'oct. 26 21:19:52 insebre java[40066]: 21:19:51.802 [stderr] [WORKER-ALPHA pid=40114 ppid=40066 thread=main] pas 1/3
d'oct. 26 21:19:52 insebre java[40066]: 21:19:52.053 [stderr] [WORKER-ALPHA pid=40114 ppid=40066 thread=main] pas 2/3
d'oct. 26 21:19:52 insebre java[40066]: 21:19:52.305 [stderr] [WORKER-ALPHA pid=40114 ppid=40066 thread=main] pas 3/3
d'oct. 26 21:19:52 insebre java[40066]: WORKER-BETA pid=40116 ppid=40066 exit=0 temps=879ms timeout=false estat=OK ordre=[/home/alumnat/.jdks/openjdk-25.0.2/bin/java, -cp, /home/alumnat/Escriptori
/2NDAM/ProgramacióDeProcessos/ProgramacioProcessos_pt1/supervisor.jar, CreacioProcessos, worker, BETA]
d'oct. 26 21:19:52 insebre java[40066]: 21:19:52.556 [stdout] COUNT=2 CHARS=8
d'oct. 26 21:19:52 insebre java[40066]: 21:19:51.797 [stderr] [WORKER-BETA pid=40116 ppid=40066 thread=main] pas 1/3
d'oct. 26 21:19:52 insebre java[40066]: 21:19:52.048 [stderr] [WORKER-BETA pid=40116 ppid=40066 thread=main] pas 2/3
d'oct. 26 21:19:52 insebre java[40066]: 21:19:52.299 [stderr] [WORKER-BETA pid=40116 ppid=40066 thread=main] pas 3/3
d'oct. 26 21:19:53 insebre java[40066]: WORKER-GAMMA pid=40163 ppid=40066 exit=0 temps=888ms timeout=false estat=OK ordre=[/home/alumnat/.jdks/openjdk-25.0.2/bin/java, -cp, /home/alumnat/Escriptor
i/2NDAM/ProgramacióDeProcessos/ProgramacioProcessos_pt1/supervisor.jar, CreacioProcessos, worker, GAMMA]
d'oct. 26 21:19:53 insebre java[40066]: 21:19:53.443 [stdout] COUNT=1 CHARS=4
d'oct. 26 21:19:53 insebre java[40066]: 21:19:52.684 [stderr] [WORKER-GAMMA pid=40163 ppid=40066 thread=main] pas 1/3
d'oct. 26 21:19:53 insebre java[40066]: 21:19:52.936 [stderr] [WORKER-GAMMA pid=40163 ppid=40066 thread=main] pas 2/3
```

```
d'oct. 26 21:29:41 insebre java[41986]: [SUPERVISOR pid=41986 ppid=1 thread=main] prova de servei llarga
d'oct. 26 21:31:11 insebre systemd[1]: practica01.service: start operation timed out. Terminating.
d'oct. 26 21:31:11 insebre java[41986]: [SUPERVISOR pid=41986 ppid=1 thread=shutdown-hook] cancel·lació: tancament del supervisor
d'oct. 26 21:31:11 insebre systemd[1]: practica01.service: Main process exited, code=exited, status=143/n/a
d'oct. 26 21:31:11 insebre systemd[1]: practica01.service: Failed with result 'timeout'.
d'oct. 26 21:31:11 insebre systemd[1]: Failed to start practica01.service - PAC1.
d'oct. 26 21:33:33 insebre systemd[1]: Starting practica01.service - PAC1...
d'oct. 26 21:33:34 insebre java[42789]: [SUPERVISOR pid=42789 ppid=1 thread=main] prova de servei llarga
d'oct. 26 21:35:03 insebre systemd[1]: practica01.service: start operation timed out. Terminating.
d'oct. 26 21:35:04 insebre java[42789]: [SUPERVISOR pid=42789 ppid=1 thread=shutdown-hook] cancel·lació: tancament del supervisor
d'oct. 26 21:35:04 insebre systemd[1]: practica01.service: Main process exited, code=exited, status=143/n/a
d'oct. 26 21:35:04 insebre systemd[1]: practica01.service: Failed with result 'timeout'.
d'oct. 26 21:35:04 insebre systemd[1]: Failed to start practica01.service - PAC1.
d'oct. 26 21:39:47 insebre systemd[1]: Starting practica01.service - PAC1...
d'oct. 26 21:39:47 insebre java[43802]: [SUPERVISOR pid=43802 ppid=1 thread=main] inici
d'oct. 26 21:39:47 insebre java[43802]: === ECHO ===
d'oct. 26 21:39:48 insebre java[43802]: ECHO pid=43823 ppid=43802 exit=0 temps=210ms timeout=false estat=OK ordre=[/home/alumnat/.jdks/openjdk-25.0.2/bin/java, -cp, /home/alumnat/Escriptori/2NDAM/
ProgramacióDeProcessos/ProgramacioProcessos_pt1/supervisor.jar, CreacioProcessos, echo]
d'oct. 26 21:39:48 insebre java[43802]: 21:39:48.130 [stdout] EBRE
d'oct. 26 21:39:48 insebre java[43802]: 21:39:48.149 [stdout] PROCESSOS
d'oct. 26 21:39:48 insebre java[43802]: 21:39:48.129 [stderr] [ECHO pid=43823 ppid=43802 thread=main] inici
d'oct. 26 21:39:48 insebre java[43802]: 21:39:48.149 [stderr] [ECHO pid=43823 ppid=43802 thread=main] final
d'oct. 26 21:39:48 insebre java[43802]: === WORKERS: TRES AMB MAXIM DOS ALHORA ===
d'oct. 26 21:39:49 insebre java[43802]: WORKER-ALPHA pid=43850 ppid=43802 exit=0 temps=956ms timeout=false estat=OK ordre=[/home/alumnat/.jdks/openjdk-25.0.2/bin/java, -cp, /home/alumnat/Escriptor
i/2NDAM/ProgramacióDeProcessos/ProgramacioProcessos_pt1/supervisor.jar, CreacioProcessos, worker, ALPHA]
d'oct. 26 21:39:49 insebre java[43802]: 21:39:49.118 [stdout] COUNT=2 CHARS=6
d'oct. 26 21:39:49 insebre java[43802]: 21:39:48.356 [stderr] [WORKER-ALPHA pid=43850 ppid=43802 thread=main] pas 1/3
d'oct. 26 21:39:49 insebre java[43802]: 21:39:48.608 [stderr] [WORKER-ALPHA pid=43850 ppid=43802 thread=main] pas 2/3
d'oct. 26 21:39:49 insebre java[43802]: 21:39:48.859 [stderr] [WORKER-ALPHA pid=43850 ppid=43802 thread=main] pas 3/3
d'oct. 26 21:39:49 insebre java[43802]: WORKER-BETA pid=43852 ppid=43802 exit=0 temps=953ms timeout=false estat=OK ordre=[/home/alumnat/.jdks/openjdk-25.0.2/bin/java, -cp, /home/alumnat/Escriptori
/2NDAM/ProgramacióDeProcessos/ProgramacioProcessos_pt1/supervisor.jar, CreacioProcessos, worker, BETA]
d'oct. 26 21:39:49 insebre java[43802]: 21:39:49.114 [stdout] COUNT=2 CHARS=8
d'oct. 26 21:39:49 insebre java[43802]: 21:39:48.351 [stderr] [WORKER-BETA pid=43852 ppid=43802 thread=main] pas 1/3
d'oct. 26 21:39:49 insebre java[43802]: 21:39:48.603 [stderr] [WORKER-BETA pid=43852 ppid=43802 thread=main] pas 2/3
d'oct. 26 21:39:49 insebre java[43802]: 21:39:48.854 [stderr] [WORKER-BETA pid=43852 ppid=43802 thread=main] pas 3/3
d'oct. 26 21:39:50 insebre java[43802]: WORKER-GAMMA pid=43899 ppid=43802 exit=0 temps=928ms timeout=false estat=OK ordre=[/home/alumnat/.jdks/openjdk-25.0.2/bin/java, -cp, /home/alumnat/Escriptor
i/2NDAM/ProgramacióDeProcessos/ProgramacioProcessos_pt1/supervisor.jar, CreacioProcessos, worker, GAMMA]
```



Es comprova que es tanquen tots els processos

6. Activitat d'anàlisi

A. **Processos o fils?**

ALPHA, BETA i GAMMA són processos independents perquè cadascun té PID propi i el seu PPID apunta al supervisor. Els noms main, pool, io-stdout i io-stderr corresponen a fils dins dels processos. El pool de 2 no crea només dos processos en total, limita a dos els workers simultanis i el tercer espera torn.

B. **Diagnosi d'errors.**

WORKER-ERROR retorna codi 7. El supervisor el representa com a ERROR i continua mostrant la resta de resultats. Una ordre inexistent és diferent: ProcessBuilder.start() falla abans de crear cap fill, amb IOException.

C. **Bloqueig evitat.**

Si el supervisor fes waitFor() abans de consumir stdout/stderr, un buffer ple podria impedir que el fill continués i també bloquejaria el pare. El mètode drain() obre un lector per cada canal abans de waitFor(); per això els dos es consumeixen concurrentment.

D. **Timeout.**

El worker LENT utilitza --sleep 2000, però el supervisor fixa un màxim o límit de 500 ms. Quan waitFor(limit) acaba l'espera, stopTree() intenta primer destroy().

E. Pipe.

El PRODUCTOR envia Ada, Pol, Joana, Nil i Marc per stdout; getPont() llegeix aquestes línies i les envia al stdin del CONSUMIDOR. En tancar el pont arriba al end of file i el consumidor retorna un recompte de: COUNT=5 CHARS=18.

F. Fork-Join.

Conceptualment el fork és pool.submit() més ProcessBuilder.start(), i el join és Process.waitFor() més Future.get(). Els workers són processos amb memòria i PID propis; no són tasques d'un ForkJoinPool.

6.1. Criteris d'acceptació: comprovació amb evidències

Criteri de l'enunciat	Estat actual	Prova o evidència
No usa shell per concatenar entrada externa	Implementat	javaCommand() + ProcessBuilder(ordre), getPont()
No deixa processos orfes	implementat	Encara viu: false.
Drena stdout i stderr sense interbloqueig	Implementat	drain() inicia dos fils lectors, resultat ECHO i workers
Errors i timeout tenen resultat explícit	Implementat	WORKER-ERROR exit=7 i WORKER-LENT timeout=true
Bateria de proves reproduïble	implementat	mvn -o test

7. Registre d'incidències

Incidència	Causa	Detecció	Solució / estat
Ordre inexistent	Ruta executable incorrecta	IOException de ProcessBuilder.start()	Registrar error i continuar la prova
Timeout worker lent	Durada superior a 500 ms	timedOut=true; Encara viu=false	stopTree() i verificar amb ps

8. Autoria, ús d'IA i verificació

Per elaborar la pràctica he utilitzat eines d'IA com a suport per entendre ProcessBuilder, els canals estàndard, la concurrència i la redacció de comentaris i documentació (javadocs). A més m'ha generat l'escript per al pstree i els tests unitàris. He revisat el codi i he comprobat tot.

També li he anat preguntant dubtes i explicacions i guies per a fer tota la tasca. M'ha ajudat també amb exemples de codi que he anat adaptant. Sobretot per al pool de 2 entre altres.